

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **SingletonPatternExample**.
2. **Define a Singleton Class:**
 - Create a class named **Logger** that has a private static instance of itself.
 - Ensure the constructor of **Logger** is private.
 - Provide a public static method to get the instance of the **Logger** class.
3. **Implement the Singleton Pattern:**
 - Write code to ensure that the **Logger** class follows the Singleton design pattern.
4. **Test the Singleton Implementation:**
 - Create a test class to verify that only one instance of **Logger** is created and used across the application.

Solution:

Logger.java:

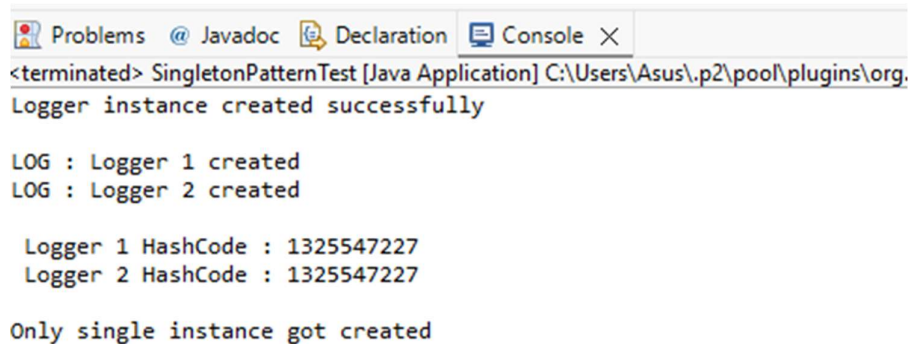
```
public class Logger {  
  
    private static Logger instance;  
  
    private Logger() {}  
  
    public static Logger getInstance() {  
        if(instance==null) {  
            instance=new Logger();  
        }  
        return instance;  
    }  
  
    public void log(String s) {
```

```
        System.out.println("LOG : "+s);
    }
}
```

SingletonPatternTest:

```
public class SingletonPatternTest {
    public static void main(String [] args) {
        Logger l1=Logger.getInstance();
        Logger l2=Logger.getInstance();
        System.out.println("Logger instance created successfully\n");
        l1.log("Logger 1 created");
        l2.log("Logger 2 created");
        System.out.println("\n Logger 1 HashCode : "+l1.hashCode());
        System.out.println(" Logger 2 HashCode : "+l2.hashCode());
        if(l1.equals(l2)) {
            System.out.println("\nOnly single instance got created");
        }
        else {
            System.out.println("\nERROR!!!! Multiple instances got created");
        }
    }
}
```

Output:



```
Problems @ Javadoc Declaration Console X
<terminated> SingletonPatternTest [Java Application] C:\Users\Asus\.p2\pool\plugins\org.
Logger instance created successfully

LOG : Logger 1 created
LOG : Logger 2 created

Logger 1 hashCode : 1325547227
Logger 2 hashCode : 1325547227

Only single instance got created
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **FactoryMethodPatternExample**.
2. **Define Document Classes:**
 - Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.
3. **Create Concrete Document Classes:**
 - Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.
4. **Implement the Factory Method:**
 - Create an abstract class **DocumentFactory** with a method **createDocument()**.
 - Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.
5. **Test the Factory Method Implementation:**
 - Create a test class to demonstrate the creation of different document types using the factory method.

SOLUTION:

Document interface and its concrete classes:

```

public interface Document {
    void open();
}

public class ExcelDocument implements Document {
    @Override
    public void open() {
        System.out.println("Opening Excel Document");
    }
}

public class PdfDocument implements Document {
    @Override
    public void open() {
        System.out.println("Opening PDF Document");
    }
}

public class WordDocument implements Document {
    @Override
    public void open() {
        System.out.println("Opening Word Document");
    }
}

```

DocumentFactory abstract class and its child classes:

```

public abstract class DocumentFactory {
    public abstract Document createDocument();
}

```

```
public class ExcelDocumentFactory extends DocumentFactory {  
    @Override  
    public Document createDocument() {  
        return new ExcelDocument();  
    }  
}
```

```
public class PdfDocumentFactory extends DocumentFactory {  
    @Override  
    public Document createDocument() {  
        return new PdfDocument();  
    }  
}
```

```
public class WordDocumentFactory extends DocumentFactory {  
    @Override  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

Implementation class(main class):

```
public class FactoryMethodTest {  
    public static void main(String[] args) {  
        DocumentFactory wordFactory = new WordDocumentFactory();  
        DocumentFactory pdfFactory = new PdfDocumentFactory();  
        DocumentFactory excelFactory = new ExcelDocumentFactory();  
  
        Document wordDoc = wordFactory.createDocument();  
        Document pdfDoc = pdfFactory.createDocument();  
    }  
}
```

```
Document excelDoc = excelFactory.createDocument();

wordDoc.open();

pdfDoc.open();

excelDoc.open();

    }

}
```

Output:

```
<terminated> FactoryMethodTest [Java Application] C:\Users\Asus\.p2\pool\plugins\org
Opening Word Document
Opening PDF Document
Opening Excel Document
```